

ANTEPROYECTO DEL TRABAJO DE FIN DE MÁSTER

INFORMACIÓN GENERAL

Alumno/a	Alejandro Román Sánchez				
Titulación:	Máster Universitario en Ingeniería del Software e Inteligencia Artificial				
Tutor/es:	José Francisco Chicano García Rubén Saborido Infantes				
Título	Refactorización de sentencias condicionales para reducir la complejidad cognitiva de código				
Subtítulo <i>(solo si es Colaborativo)</i>					
Título en inglés	Refactoring conditional sentences to reduce code cognitive complexity				
Subtítulo en inglés <i>(solo si es Colaborativo)</i>					
Trabajo colaborativo:	Sí	☐	No	☒	
Otros integrantes del grupo:					
Idioma de la memoria y defensa:	Castellano	☒	Inglés	☐	

INTRODUCCIÓN

Contextualización del problema a resolver. Descripción detallada de la motivación de este TFM y el dominio de aplicación.

Una de las fases por la que pasan todos los proyectos de desarrollo de software en la industria es la de mantenimiento, donde el código es modificado para incorporar nueva funcionalidad, resolver defectos o modificar dependencias problemáticas. De cara a esta fase de mantenimiento, es importante que el código sea fácil de comprender por los desarrolladores, que pueden ser los mismos o diferentes a los que escribieron originalmente el código. Comprender el código fuente influye en su evolución y mantenimiento. A pesar de que lo anterior es un aspecto de la calidad interna de todo código fuente, en ocasiones, se descuida debido a la presión por terminar el trabajo a tiempo o con unos determinados recursos. La literatura está llena de métricas definidas para medir la mantenibilidad del código. En este trabajo nos centraremos en una de ellas: la complejidad cognitiva.

Existen diversas definiciones de complejidad cognitiva, de las cuales escogeremos la definida por SonarSource, la empresa detrás de productos como SonarQube, que se ha convertido en un estándar de facto en el mundo de desarrollo software para medir la calidad del código. Esta métrica se define a nivel de método y su valor es un número entero que depende de la presencia de instrucciones de control de flujo y su anidamiento dentro de otras instrucciones. SonarSource recomienda que el valor de esta métrica no sea mayor que 15 en ningún método, e incluye esta recomendación como regla en su herramienta SonarQube.

Una forma de reducir la complejidad cognitiva consiste en aplicar extracciones de líneas de código, para formar nuevos métodos que serán llamados desde el método original donde inicialmente se encontraban las líneas de código extraídas. Es decir, se trata de aplicar la refactorización de extracción de método al código para mantener su semántica a la vez que se reduce la complejidad cognitiva del método original. En trabajos recientes del grupo de investigación NEO se ha comprobado que en ocasiones se encuentran en el código sentencias condicionales anidadas que poseen una gran complejidad cognitiva pero que pueden convertirse en una única sentencia condicional con menor complejidad cognitiva combinando las expresiones lógicas asociadas a las sentencias.

OBJETIVOS

Descripción detallada de este TFM. Preguntas de investigación a abordar (si procede).

Este TFM plantea investigar formas de refactorizar las sentencias condicionales de código Java para reducir la complejidad cognitiva del mismo. Las preguntas de investigación que nos planteamos son:

RQ1: ¿En qué casos se pueden combinar sentencias condicionales anidadas y cuál sería el resultado?

RQ2: ¿Qué impacto tienen estas refactorizaciones en la complejidad cognitiva?

RQ3: ¿Pueden los grandes modelos de lenguaje realizar esta refactorización de forma correcta y automática?

ENTREGABLES

Listado de resultados que generará el TFM (aplicaciones, estudios, manuales, etc.)

Memoria del TFM

Herramienta o prototipo de análisis/refactorización

Paquete de replicación

Manual técnico

MÉTODOS Y FASES DE TRABAJO**METODOLOGÍA:**

Descripción de la metodología empleada en el desarrollo del TFM. Especificar cómo se va a desarrollar. Concretar si se trata de alguna metodología existente. En caso contrario, describir y justificar adecuadamente los métodos que se aplicarán.

Para el desarrollo de este TFM se seguirá una metodología de investigación científica aplicada enmarcada en Design Science Research (DSR), adecuada cuando el objetivo es diseñar y validar un artefacto técnico. En este contexto, el artefacto consiste en un conjunto de reglas de refactorización, con sus precondiciones de aplicabilidad y un prototipo capaz de detectarlas y aplicarlas sobre código Java. La investigación se guiará por las preguntas de investigación planteadas y se sustentará en un ciclo de construcción y evaluación: las reglas se diseñarán y formalizarán a partir de la evidencia del estado del arte y de casos observados, y se validarán mediante un protocolo empírico reproducible.

La evaluación considerará tanto el impacto, variación de complejidad cognitiva según SonarSource, como la corrección, preservación semántica mediante compilación y ejecución de pruebas cuando sea posible. Adicionalmente, el mismo protocolo permitirá una comparación controlada con refactorizaciones generadas por grandes modelos de lenguaje, aplicando métricas y criterios de validez equivalentes.

FASES DE TRABAJO:

Enumeración y breve descripción de las fases de trabajo para el desarrollo del TFM.

- Revisión del estado del arte – Estudio de la complejidad cognitiva y de refactorizaciones relacionadas con condicionales, incluyendo enfoques existentes para reescrituras semánticamente equivalentes.
- Definición del alcance y requisitos del estudio – Concreción de qué estructuras de Java se cubrirán qué se considera “condicional anidada combinable”, criterios de aplicabilidad y criterios de corrección semántica.
- Diseño del enfoque de refactorización – Especificación de patrones y reglas de transformación, incluyendo precondiciones de seguridad, casos no aplicables y ejemplos antes/después.
- Construcción del *dataset* de evaluación – Selección y preparación de un conjunto de métodos Java con condicionales anidados, con metadatos y, si procede, refactorizaciones de referencia.
- Desarrollo del prototipo/herramienta – Implementación de un sistema para detectar oportunidades y aplicar automática o semiautomáticamente la combinación de condicionales sobre el código.
- Medición y evaluación del impacto – Ejecución de experimentos para medir complejidad cognitiva antes/después y analizar resultados.
- Validación de corrección semántica – Verificación del comportamiento del código refactorizado mediante compilación y, cuando sea posible, ejecución de pruebas; registro y análisis de fallos.

- Evaluación con grandes modelos de lenguaje (LLM) – Definición de *prompts* y procedimiento experimental, generación de refactorizaciones con LLM y comparación con el prototipo/reglas usando las mismas métricas.
- Análisis de resultados y amenazas a la validez – Interpretación de hallazgos respecto a RQ1–RQ3, discusión de limitaciones y propuesta de mejoras.
- Documentación y entrega final – Redacción de la memoria, generación del paquete de replicación y preparación de la defensa.

ENTORNO TECNOLÓGICO

TECNOLOGÍAS EMPLEADAS:

Enumeración de las tecnologías utilizadas (lenguajes de programación, frameworks, sistemas gestores de bases de datos, etc.) en el desarrollo del TFM.

Se utilizará Java como lenguaje principal de programación.

Git y Github como herramienta de control de versiones.

Maven o Gradle para la gestión de dependencias y automatización de build/test

Parser/AST para Java como librería para analizar el código fuente a nivel de árbol sintáctico. [1]

SonarQube/SonarLint como herramienta de análisis estático para medir la complejidad cognitiva. [2]

API/SDK del modelo de LLM para la integración para generar refactorizaciones automáticas a través de prompts.

RECURSOS SOFTWARE Y HARDWARE:

Listado de dispositivos (placas de desarrollo, microcontroladores, procesadores, sensores, robots, etc.) o software (IDE, editores, etc.) empleados en el desarrollo del TFM.

IDE IntelliJ Community Edition

JDK (Java 11/17)

SonarQube y/o SonarLint [2]

Latex para la realización de la memoria

CONJUNTOS DE DATOS (DATASETS):

Listado de los conjuntos de datos (datasets) que serán empleados en el desarrollo del TFM (si procede), detallando en su caso su procedencia y licencia de uso.

Se partirá de los 10 proyectos open-source recopilados en el repositorio SoftwareCognitiveComplexityReducer [3]. El conjunto incluye del orden de 1.000 métodos con CC por encima del umbral recomendado por Sonar (CC>15). Se respetarán las licencias de cada proyecto y del repositorio, documentándolas en el paquete de replicación. Adicionalmente, el umbral de CC se considerará un parámetro experimental y podrá ajustarse para ampliar la muestra.

Se contempla además de la posibilidad de incorporar proyectos adicionales para mejorar la generalización de las conclusiones.

REFERENCIAS

Listado de referencias (artículos científicos, libros, páginas web, etc.)

- [1] JavaParser, «JavaParser — Home,» JavaParser, [En línea]. Available: <https://javaparser.org/>. [Último acceso: 27 12 2025].
- [2] SonarSource, «SonarQube,» SonarSource, [En línea]. Available: <https://www.sonarsource.com/products/sonarqube/>. [Último acceso: 27 12 2025].
- [3] R. Saborido, J. Ferrer, F. Chicano and E. Alba, "Automatizing Software Cognitive Complexity Reduction" in IEEE Access, vol. 10, pp. 11642-11656, 2022, doi: 10.1109/ACCESS.2022.3144743.



UNIVERSIDAD
DE MÁLAGA



Málaga, 09 de febrero de 2026

Firma tutor/tutora:

Firma cotutor/a: